

컴파일된 위키와 SQLite 테이블이 커질 때 발생하는 토큰 과소비와 응답 지연 문제는 충분히 우려할 만한 부분입니다. 다행히도 이는 현대 RAG 및 Text-to-SQL 시스템에서 이미 잘 연구된 분야이며, KDQE 구조 내에서도 효과적인 최적화 전략을 세울 수 있습니다.

🗨️ 1. 비정형 지식 (OKF 위키) 검색 최적화

OpenKB가 커질수록 LLM에 모든 위키 내용을 전달하는 것은 비효율적입니다. 다음 전략들을 고려하세요.

- **검색 단계의 강화 (Retrieval Augmentation)**: 모든 위키를 LLM에 바로 전달하는 대신, **검색(Retrieval)** 단계를 거쳐 질문과 관련된 소수의 페이지만 추출합니다. OpenKB는 `PageIndex` 라는 자체 검색 메커니즘을 갖추고 있어, 이의 정확도를 높이는 방향으로 튜닝할 수 있습니다.
- **하이브리드 검색 (Hybrid Search)**: 의미론적 검색(Semantic Search)과 키워드 검색(Keyword Search)을 결합합니다. 벡터 DB와 SQLite의 FTS5(Full-Text Search)를 함께 사용해 검색의 정확도와 재현율을 높일 수 있습니다.
- **청크 최적화 (Chunking Optimization)**: 위키 페이지를 LLM에 전달할 때는 적절한 크기로 나누는 것이 중요합니다. 너무 크면 토큰을 낭비하고, 너무 작으면 문맥이 끊깁니다.
- **컨텍스트 압축 (Context Compression)**: 검색된 관련 문서라도 모든 내용을 그대로 전달하기보다는, **요약**하거나 질문과 관련 없는 부분을 제거하는 프롬프트 압축 기술을 적용해 토큰 사용량을 크게 줄일 수 있습니다.

🗄️ 2. 정형 데이터 (SQLite) 질의 최적화

테이블이 많아질수록 전체 스키마를 LLM에 전달하는 것은 비용과 성능 측면에서 치명적입니다.

- **동적 스키마 링킹 (Dynamic Schema Linking)**: 전체 스키마 대신, **질문과 관련된 테이블과 컬럼만**을 선택적으로 LLM에 제공하는 것이 핵심입니다.
 - **FAISS 유사도 기반 검색**: 관련성이 높은 스키마 요소를 찾아내는 방법.
 - **외래키 그래프 탐색 (FK-Graph Expansion)**: 관련 테이블을 연결하는 방법.
 - **최적 사례**: 이러한 기법들을 적용하면 프롬프트 토큰을 **최대 95% 이상** 절감할 수 있습니다. 17k 토큰이 100토큰 미만으로 줄어든 사례도 있습니다.
- **의미 계층(Semantic Layer)의 활용**: OKF가 바로 이 역할을 합니다. `VIP 고객` 과 같은 비즈니스 용어를 미리 정의해 두면, 매번 복잡한 SQL을 생성할 필요 없이 정의된 로직을 재사용할 수 있습니다. 또한, 스키마 정보를 YAML 등으로 구조화하면 토큰 사용량을 **20~30%** 절감할 수 있습니다.

⚙️ 3. 통합 질의 및 아키텍처 최적화

비정형과 정형 데이터를 함께 다루는 KDQE의 특성상, 전체적인 조율이 중요합니다.

- **질의 계획(Query Planning) 최적화**: 실행 계획을 세울 때, **비용 기반 최적화(Cost-based Optimization)** 개념을 도입합니다. 예를 들어, 집계(Count)가 필요한 질문이라면 관련 문서를 모두 검색(RAG)하기보다는 SQLite에 먼저 질의하는 것이 훨씬 효율적일 수 있습니다.
- **캐싱(Caching) 전략**:
 - **결과 캐싱**: 동일하거나 유사한 질문이 들어오면 이전 실행 결과를 재사용합니다.
 - **스키마 캐싱**: 자주 사용되는 스키마 정보나 OKF 정의는 메모리에 캐싱해 매번 조회하는 오버헤드를 줄입니다.
- **작은 모델의 활용 (Fine-Tuning)**: Text-to-SQL 생성이나 문서 요약과 같은 반복적인 작업은 **전문적으로 파인튜닝된 소형 로컬 모델**에 맡기는 것도 좋은 방법입니다. 이렇게 하면 값비싼 대형 모델 호출을 줄일 수 있습니다.

💡 정리: 단계별 적용 로드맵

이러한 최적화 전략들을 KDQE에 단계별로 적용해 보세요.

1. **1단계: 스키마 링킹 고도화**: Text-to-SQL 단계에서 동적 스키마 링킹을 가장 먼저 도입합니다. 이는 가장 확실한 토큰 절감 효과를 줍니다.
2. **2단계: 검색 파이프라인 개선**: OKF 위키 검색에 하이브리드 검색을 도입하고, 청크 크기와 검색 결과 개수(top-k)를 조정합니다.
3. **3단계: 컨텍스트 압축 및 캐싱**: LLM에 전달되는 최종 컨텍스트에 압축을 적용하고, 자주 쓰이는 질의와 결과는 캐싱합니다.
4. **4단계: 비용 기반 질의 플래너 구현**: 정형/비정형 데이터 소스 중 어디에 먼저 질의할지를 결정하는 지능적인 플래너를 구축합니다.

이러한 최적화 전략들은 이미 많은 상용 시스템에서 검증된 방법들입니다. KDQE 구조에 맞게 단계적으로 적용한다면, 지식 베이스와 데이터베이스가 성장해도 비용과 성능을 안정적으로 관리할 수 있을 것입니다.

OpenKB의 PageIndex는 **파일 단위**로 동작하지만, 내부적으로는 문서의 구조를 **계층적 트리**로 인덱싱하는 방식입니다.

🧠 PageIndex의 인덱싱 단위

문서의 길이에 따라 인덱싱 방식이 달라집니다:

- **짧은 문서 (예: 20페이지 미만)**: 문서 **전체**를 하나의 단위로 간주하여 LLM이 전문을 직접 읽습니다.
- **긴 문서 (예: 20페이지 이상의 PDF)**: 문서 **자체**를 **계층적 트리(Tree)**로 인덱싱합니다. 책의 '목차 - 챕터 - 절'처럼 문서를 **의미 있는 단위(섹션, 챕터)**로 나누고, 각 단위의 **요약**을 **계층적으로 구성**합니다. 이렇게 하면 LLM이 전문을 읽는 대신 이 **요약 트리**를 읽어 문서의 전체적인 흐름과 맥락을 파악할 수 있습니다.

📁 소스 문서 구성 전략

PageIndex가 효과적으로 작동하려면, 문서를 **명확한 계층 구조**로 작성하는 것이 중요합니다.

- **구조화된 문서 작성**: 문서에 **명확한 제목(Headings)**과 **부제목(Sub-headings)**을 사용해 섹션을 논리적으로 구분하세요. 이 구조가 PageIndex가 트리를 만드는 기본 뼈대가 됩니다.
- **의미 단위로 분할**: 하나의 문서가 너무 방대해지기보다는, **독립적인 주제로** 문서를 나누는 것도 좋은 방법입니다. 예를 들어, '제품 정책', '고객 정책'처럼 말이죠.
- **문서 간 연결 활용**: OpenKB는 위키처럼 문서 간 링크를 지원합니다. 문서 내에서 관련 개념이나 다른 문서를 **[개념명](경로)** 형식으로 연결하면, 지식 그래프가 강화되어 검색 품질이 높아집니다.

💡 검색 및 응답 생성 과정

1. **질문 접수**: 사용자가 자연어로 질문합니다.
2. **트리 탐색**: PageIndex는 질문과 관련된 부분을 찾기 위해 **계층적 요약 트리**를 탐색합니다. 마치 책의 목차를 보고 필요한 챕터를 찾는 것과 같습니다.
3. **관련 청크 식별**: 관련성이 높은 하위 섹션이나 특정 부분을 찾아냅니다.
4. **응답 생성**: LLM은 찾아낸 관련 부분들의 **원본 텍스트**를 읽고, 이를 바탕으로 사용자에게 최종 답변을 생성합니다.

즉, PageIndex는 **문서 전체를 하나의 덩어리로 보지 않고, 구조화된 지식 지도로 변환하여** 효율적인 탐색과 검색을 가능하게 합니다. 따라서 소스 문서에 명확한 계층 구조를 부여하는 것이 성능 향상의 핵심입니다.

한두 개의 테이블로 제한한다면, '**범용 엔티티-속성-값(EAV) 모델**' 또는 '**문서형 스키마**'를 채택하는 것이 가장 효율적입니다. 이렇게 하면 테이블 수는 적지만 다양한 도메인의 데이터를 수용할 수 있습니다.

📊 권장 테이블 구성 (2개 테이블)

1. `entities` 테이블 (주요 엔티티 저장)

```
CREATE TABLE entities (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  entity_type TEXT NOT NULL,      -- 'customer', 'product', 'order', 'employee' 등  
  name TEXT NOT NULL,            -- 엔티티 이름  
  attributes JSON,               -- 유연한 속성 저장 (JSON)  
  metadata JSON,                 -- 생성일, 수정일 등 메타데이터  
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
  updated_at DATETIME DEFAULT CURRENT_TIMESTAMP  
);  
  
-- 인덱스  
CREATE INDEX idx_entity_type ON entities(entity_type);  
CREATE INDEX idx_entity_name ON entities(name);
```

2. relations 테이블 (엔티티 간 관계 저장)

```
CREATE TABLE relations (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  source_id INTEGER NOT NULL,      -- entities.id (출발 엔티티)  
  target_id INTEGER NOT NULL,      -- entities.id (도착 엔티티)  
  relation_type TEXT NOT NULL,      -- 'belongs_to', 'purchased', 'manages' 등  
  attributes JSON,                 -- 관계 속성 (예: 구매일, 금액 등)  
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (source_id) REFERENCES entities(id),  
  FOREIGN KEY (target_id) REFERENCES entities(id)  
);  
  
-- 인덱스  
CREATE INDEX idx_relation_source ON relations(source_id);  
CREATE INDEX idx_relation_target ON relations(target_id);  
CREATE INDEX idx_relation_type ON relations(relation_type);
```

데이터 삽입 예시

```
-- 고객 추가  
INSERT INTO entities (entity_type, name, attributes) VALUES  
('customer', 'John Doe', '{"email": "john@example.com", "age": 30, "vip": true, "total_purchases": 5}');  
  
-- 제품 추가  
INSERT INTO entities (entity_type, name, attributes) VALUES  
('product', 'Laptop Pro X', '{"category": "electronics", "price": 1200000, "stock": 50}');  
  
-- 관계 추가 (고객이 제품을 구매)  
INSERT INTO relations (source_id, target_id, relation_type, attributes) VALUES  
(1, 2, 'purchased', '{"quantity": 1, "purchase_date": "2026-01-15", "amount": 1200000}');
```

SKILL.md 구성 (Hermes Skill)

에이전트가 이 테이블 구조를 이해하고 질의할 수 있도록 SKILL.md를 작성합니다.

```
---  
name: ecommerce-query  
description: SQLite 기반 e-커머스 데이터 질의 스킬  
version: 1.0.0  
tools:  
  - sqlite_query  
---
```

E-Commerce Data Query Skill

🇰🇷 데이터베이스 스키마

이 스킬은 다음 2개 테이블로 구성된 SQLite 데이터베이스를 질의합니다.

1. entities 테이블
범용 엔티티 저장소입니다.

```
| 컬럼 | 타입 | 설명 |
|-----|-----|-----|
| id | INTEGER | PK |
| entity_type | TEXT | 'customer', 'product', 'order', 'employee' |
| name | TEXT | 엔티티 이름 |
| attributes | JSON | 유연한 속성 (email, age, vip, price, stock 등) |
| created_at | DATETIME | 생성일 |
```

2. relations 테이블
엔티티 간 관계를 저장합니다.

```
| 컬럼 | 타입 | 설명 |
|-----|-----|-----|
| id | INTEGER | PK |
| source_id | INTEGER | 출발 엔티티 (FK → entities.id) |
| target_id | INTEGER | 도착 엔티티 (FK → entities.id) |
| relation_type | TEXT | 'purchased', 'belongs_to', 'manages' |
| attributes | JSON | 관계 속성 (quantity, amount, purchase_date) |
```

🔍 질의 패턴

고객 정보 조회

```
```sql
SELECT * FROM entities
WHERE entity_type = 'customer'
AND json_extract(attributes, '$.email') = 'john@example.com';
```

## VIP 고객 목록

```
SELECT name, json_extract(attributes, '$.total_purchase') as total
FROM entities
WHERE entity_type = 'customer'
AND json_extract(attributes, '$.vip') = true;
```

## 특정 고객의 구매 이력

```
SELECT
 c.name as customer_name,
 p.name as product_name,
 r.attributes->>'$.amount' as amount,
 r.attributes->>'$.purchase_date' as purchase_date
FROM entities c
JOIN relations r ON r.source_id = c.id
JOIN entities p ON r.target_id = p.id
WHERE c.entity_type = 'customer'
AND c.name = 'John Doe'
AND r.relation_type = 'purchased';
```

## 카테고리별 매출 집계

```

SELECT
 json_extract(p.attributes, '$.category') as category,
 SUM(json_extract(r.attributes, '$.amount')) as total_sales
FROM entities p
JOIN relations r ON r.target_id = p.id
WHERE p.entity_type = 'product'
 AND r.relation_type = 'purchased'
GROUP BY category;

```

## 🎯 OKF 개념 매핑

비즈니스 용어	SQL 매핑
VIP 고객	<code>entity_type='customer' AND json_extract(attributes, '\$.vip')=true</code>
매출	<code>SUM(json_extract(r.attributes, '\$.amount'))</code>
전자제품	<code>entity_type='product' AND json_extract(attributes, '\$.category')='electronics'</code>
최근 3개월	<code>r.attributes-&gt;'\$.purchase_date' &gt;= date('now', '-3 months')</code>

## 📌 사용 예시

질문: "VIP 고객의 평균 구매액은?" SQL:

```

SELECT AVG(json_extract(r.attributes, '$.amount')) as avg_purchase
FROM entities c
JOIN relations r ON r.source_id = c.id
WHERE c.entity_type = 'customer'
 AND json_extract(c.attributes, '$.vip') = true
 AND r.relation_type = 'purchased';

```

질문: "가장 많이 팔린 제품 TOP 5는?" SQL:

```

SELECT
 p.name,
 SUM(json_extract(r.attributes, '$.quantity')) as total_sold
FROM entities p
JOIN relations r ON r.target_id = p.id
WHERE p.entity_type = 'product'
 AND r.relation_type = 'purchased'
GROUP BY p.id
ORDER BY total_sold DESC
LIMIT 5;

```

## ⚠️ 주의사항

1. JSON 필드는 `json_extract()` 함수로 접근
2. 날짜 비교는 ISO 형식('YYYY-MM-DD') 사용
3. 대량 데이터 조회 시 `LIMIT` 절 필수
4. 관계 조회 시 `source_id` 와 `target_id` 방향 확인

## 🔧 확장 팁

- `entity_type` 을 추가하면 새로운 도메인(예: 'employee', 'store') 확장 가능
  - `relation_type` 을 추가하면 새로운 관계(예: 'manages', 'reviews') 확장 가능
  - JSON 필드에 인덱스를 걸려면 `GENERATED COLUMN` 활용
- 

## 💡 KDQE와의 통합

---

이 SKILL.md를 Hermes Skill로 등록하면:

1. **OKF Resolver**가 `entities` , `relations` 테이블을 Concept으로 인식
2. **Question Analyzer**가 "VIP 고객", "매출" 등의 용어를 OKF Concept과 매핑
3. **Plan Generator**가 SKILL.md의 SQL 패턴을 참조해 실행 계획 수립
4. **Executor**가 생성된 SQL을 SQLite에서 실행

이렇게 하면 테이블이 2개로 제한되어도 **다양한 비즈니스 질문에 유연하게 대응**할 수 있습니다. JSON 필드를 활용하면 엔티티 타입을 추가할 때마다 테이블을 새로 만들 필요가 없어 유지보수도 쉬워집니다.